often in the standard library or in the wider ecosystem (third-party packages, modules, or libraries).

The second main benefit of logs is that, due to their textual nature, a human can usually interpret the meaning; however, there is still a requirement for a developer to be able to write actionable log lines (https://logz.io/blog/logging-best-practices/), such as formatting, context, and so on. Beyond that, all you need is to find a log line, and you can interpret it. There are two approaches to finding relevant log lines:

- Traditionally, every log line (or log item) was *full-text indexed*. Systems like Elasticsearch or OpenSearch as well as many commercial offerings take this approach. This is great, since you can search for everything; however, it comes with scalability and cost challenges.
- Recently, *label-based indexing* has become popular. Rather than indexing the entire text of a log line, you only need to index the labels (or tags) that provide the context (e.g., this is from service `foo`, and it is about and `error` condition). The most popular example of this approach is Grafana Loki (https://grafana.com/oss/loki/), inspired by Prometheus.

With this, we've reached the end of our logs cost and benefit discussion, so let's move on to use cases for logs in the context of observability.

2.3.4 Observability with logs

What can we do with logs, and how is what we just saw in the hands-on part related to observability?

As a developer, you can use logs

- For debugging an existing code base (for which you either own the source code or can reverse engineer it).
- To perform testing (e.g., soak tests, performance tests, or chaos engineering) and see where and how your microservices break apart.

Further, in a DevOps or SRE role, logs

- Help identify the service and code responsible for a bug or outage with the goals to (a) stop the bleeding (restart or take it offline) and (b) fix the issue in the code identified.
- Provide context (e.g., identify who made which change when) as well as address regulatory or security-related concerns. The Log4j vulnerability CVE-2021-44228 (https://nvd.nist.gov/vuln/detail/CVE-2021-44228) and CVE-2021-45046 (https://nvd.nist.gov/vuln/detail/CVE-2021-45046), which we saw at the end of 2021, are good examples of security-motivated use cases.

2.4 Metrics

Metrics are numerical signals, typically sampled at regular time intervals. They usually consist of a name, a value, and some metadata. The name identifies the metric (e.g.,